

gave me downloadable file which contains all the notes and gave me handwritten notes
dents skip any thing DSA Basics — Day 1

Programming Fundamentals: C++ & Python

1. Course Goal

- This is **Day 1 of DSA Basics in 10 Days**.
 - The aim is to learn enough programming fundamentals to move toward **DSA patterns**.
 - The same basic syllabus is covered in:
 - **C++**
 - **Java**
 - **Python**
 - If you have no language preference, the lecture recommends **C++**, as it is the instructor's strongest language.
 - The course starts from the absolute basics, so prior DSA/CS knowledge is not required.
-

PART A — C++ BASICS

2. What is a Programming Language?

Computer Language

- Computers fundamentally understand **binary: 0 and 1**.
- Writing binary directly is difficult for humans.
- Programming languages provide a more human-readable way to communicate with computers.
- Coding languages are presented as a combination of concepts close to **English + Mathematics**.

Example:

```
int a = 10;
```

This is easier for humans to write than directly writing binary.

3. C++ Program Template

A basic C++ program uses a standard template:

```
#include <bits/stdc++.h>
using namespace std;

int main()
{
    // Your code goes here

    return 0;
}
```

Important

- Initially, you don't need to understand every line.
- Treat this as your **C++ template**.
- For the upcoming programs, copy the template and write your actual code inside `main()` .

Learning strategy: First learn to *write and execute* programs; deeper language details can be understood gradually.

4. Output / Printing in C++

C++ uses:

```
cout << "Hello World";
```

Rules

- Text/constants are written inside " " .
- << is part of the syntax.
- A statement generally ends with ; .

Example

```
cout << "Hello DSA";
```

Output:

```
Hello DSA
```

To print something else, simply change the text inside the quotes.

5. Input in C++

C++ uses:

```
cin >> a;
```

Remember

- cout **Output**
- cin **Input**

The direction of the arrows is important:

```
cout << a;  
cin >> a;
```

6. Variables

A **variable** is a named location used to store a value.

This is similar to variables in mathematics:

```
x = 5
```

Constant vs Variable

Concept	Example
Constant	10 , 5
Variable	a , x , b

7. Data Types

Every variable has a **data type**.

Example:

```
int a = 10;
```

`int` means **integer**.

Therefore, `a` is intended to store an integer value.

Examples:

```
int a = 10;
int b = 20;
```

You can later change the value:

```
a = 5;
```

Key Point

If a variable is declared as `int`, the lecture focuses on using it for **integer values**.

Other data types mentioned for later lessons:

- `string`
 - `char`
 - `float`
 - `double`
-

8. Printing Variables

To print a constant:

```
cout << "Hello";
```

To print a variable:

```
cout << a;
```

Important Difference

```
cout << a;
```

prints the **value stored in** `a`

```
cout << "a";
```

prints the literal character/text **a**

Example

```
int a = 10;  
cout << a;
```

Output:

```
10
```

9. Printing Text + Variable

You can combine a constant string and a variable:

```
cout << "Hello " << a;
```

If:

```
a = 10;
```

Output:

```
Hello 10
```

The text inside quotes remains constant, while **a** contributes its current value.

10. New Line — endl

To move output to the next line:

```
cout << "Hello" << endl;  
cout << "DSA";
```

Output:

```
Hello  
DSA
```

Remember

endl = end line

Use it when you want the next output to appear on a new line.

11. Basic Operators in C++

The lecture introduces five basic arithmetic operators:

Operator	Meaning
+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Modulus / Remainder

12. Addition Example

```
int a = 5;  
int b = 10;  
  
cout << a + b;
```

Output:

```
15
```

You can also store the result:

```
int c = a + b;  
cout << c;
```

Here:

```
a = 5  
b = 10  
c = 15
```

13. Hard-Coded vs Dynamic Program

Hard-coded

```
int a = 5;  
int b = 10;
```

Every time you run the program, the same values are used.

Dynamic/Input-based

```
int a;  
int b;  
  
cin >> a;  
cin >> b;  
  
cout << a + b;
```

Now the user can provide different values every time.

Key Idea

Input makes a program dynamic.

14. Division /

Division has an important behavior with integers.

Example:

```
int a = 5;  
int b = 2;  
  
cout << a / b;
```

The result is:

```
2
```

—not 2.5 , because the variables are integers.

Quotient vs Remainder

For:

$$5 \div 2$$

- Quotient = 2
- Remainder = 1

The `/` operator gives the **quotient** when working with integers.

15. Modulus %

The `%` operator gives the **remainder**.

Examples

$$4 \% 2$$

Result:

$$0$$

Because 4 is perfectly divisible by 2.

$$5 \% 2$$

Result:

$$1$$

Because the remainder is 1.

Easy Memory Trick

/ Quotient
% Remainder

16. C++ Error & Debugging

Errors are a normal part of programming.

Example: forgetting a semicolon:

```
int a = 5
```

The compiler reports an error because the statement needs `;`.

Correct:

```
int a = 5;
```

Mindset

Don't be afraid of errors.

Errors are feedback that helps you debug your program.

PART B — PYTHON BASICS

17. What is Python?

- Python is described in the lecture as an **interpreted language**.
- It is considered a **high-level language** because its syntax is relatively close to human language.
- The focus here is not on learning every feature of Python, but on learning the basics required for DSA.

18. Compiled vs Interpreted — Basic Idea

Compiled

The lecture describes C++/Java as compiled languages.

The complete program is checked before execution.

Interpreted

Python is described as interpreted.

The program is executed **line by line**.

Example

Suppose:

```
print("Hello")  
print("DSA")  
print(error)
```

The first two lines can execute before an error is encountered on the third line.

Memory Trick

Python interpreted line-by-line execution

19. Printing in Python

Python uses:

```
print("Hello World")
```

Example:

```
print("Hello DSA")
```

Output:

```
Hello DSA
```

Difference from C++

Python does not require the C++ style program template.

A simple Python program can be just:

```
print("Hello World")
```

20. Semicolon in Python

Python does **not require** a semicolon at the end of a statement.

Example:

```
print("Hello")
```

is sufficient.

The lecture notes that people coming from C/C++/Java may have a habit of writing semicolons, but Python does not require them for ordinary statements.

21. Variables in Python

Python makes variable declaration simple:

```
a = 10
```

No explicit `int` declaration is required here.

The variable name can be changed:

```
b = 10  
c = 10
```

A variable can also store text:

```
a = "Hello"
```

Then:

```
print(a)
```

prints:

```
Hello
```

22. Printing Variables in Python

Constant

```
print("a")
```

Output:

```
a
```

Variable

```
a = 10  
print(a)
```

Output:

```
10
```

Key Difference

Quotes mean:

| *Print exactly what is written.*

No quotes around a variable mean:

| *Print the value stored in that variable.*

23. Taking Input in Python

Basic input:

```
a = input("Enter value of A: ")
```

The user can enter a value when the program runs.

Then:

```
print(a)
```

prints the entered value.

24. Important: Python `input()` Returns a String

This is one of the most important concepts from the lecture.

Suppose:

```
a = input()
b = input()

print(a + b)
```

If the user enters:

```
12
34
```

The output is:

```
1234
```

Why?

Python treats values received through `input()` as **strings**.

So:

```
"12" + "34"
```

means string concatenation:

```
"1234"
```

It does **not** perform numerical addition.

25. Converting Input to Integer

If the input should be treated as a number:

```
a = int(input("Enter value of A: "))  
b = int(input("Enter value of B: "))
```

Now:

```
print(a + b)
```

If the inputs are:

```
30  
40
```

Output:

```
70
```

Remember

```
input()
```

gives input as a string

```
int(input())
```

converts the input into an integer

26. Python Operators

The same basic mathematical operators are used:

Operator	Meaning
+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Modulus / Remainder

Example:

```
a = 10
b = 5

c = a + b
print(c)
```

Output:

```
15
```

27. Printing String + Variable in Python

Python allows multiple values to be passed to `print()` using commas.

Example:

```
print("Sum of A and B is", c)
```

If:

```
c = 30
```

Output:

```
Sum of A and B is 30
```

This is useful when you want to display both **text and calculated values**.

28. Comments in C++

Comments are ignored by the compiler.

Single-line comments use:

```
// This is a comment
```

Anything after `//` on that line is not executed.

Comments are useful for:

- Explaining code
 - Writing notes
 - Making code easier to understand
-

29. C++ vs Python — Quick Comparison

Feature	C++	Python
Basic print	<code>cout <<</code>	<code>print()</code>
Input	<code>cin >></code>	<code>input()</code>
Variable example	<code>int a = 10;</code>	<code>a = 10</code>
Semicolon	Generally required	Not required
Basic string printing	<code>"Hello"</code>	<code>"Hello"</code>
Integer input	<code>cin >> a</code> with <code>int a</code>	<code>int(input())</code>
Language description in lecture	Compiled	Interpreted
Syntax level	More verbose	More concise

30. Most Important Concepts to Remember

C++

`cout` output
`cin` input
`int` integer data type
`;` statement termination
`endl` next line

Python

`print()` output
`input()` input as string

```
int(input())    integer input
```

Operators

```
+    addition
-    subtraction
*    multiplication
/    quotient/division
%    remainder
```

Variables

```
C++    int a = 10;
Python a = 10;
```

Quotes

```
"a"    literal text
a      value stored in variable a
```

31. Problem-Solving Pattern

A large number of beginner programs follow this simple structure:

Step 1 — Take Input

```
Input    a, b
```

Step 2 — Perform Operation

```
c = a + b
```

Step 3 — Print Result

```
Output  c
```

This pattern is the foundation for many basic programming problems.

32. Homework / Practice Questions

Q1. Product of Three Numbers

Take **three numbers as input** and print their product.

Concept:

```
a × b × c
```

Q2. Average of Two Numbers

Take **two numbers as input** and print their average.

Basic idea:

```
Average = Total / 2
```

For two numbers:

```
Average = (a + b) / 2
```

When implementing this in a language, pay attention to the data type used for the result.

33. Final Revision Sheet

C++ Template

```
#include <bits/stdc++.h>
using namespace std;

int main()
{
    // code

    return 0;
}
```

C++ Print

```
cout << "Hello";
```

C++ Input

```
cin >> a;
```

C++ Variable

```
int a = 10;
```

C++ Arithmetic

```
a + b  
a - b  
a * b  
a / b  
a % b
```

Python Print

```
print("Hello")
```

Python Variable

```
a = 10
```

Python Input

```
a = input()
```

Python Integer Input

```
a = int(input())
```

Python Arithmetic

```
c = a + b
```

Python Output

```
print(c)
```

One-Line Takeaway

Day 1 teaches the fundamental programming cycle:

Input Store in Variables Perform Operations Output the Result

Master this pattern first; it becomes the base for solving DSA problems later. DSA BASICS
— DAY 2

If-Else & Loops | C++ • Java • Python

1. What You Learn Today

Today's lecture focuses on two **fundamental programming concepts**:

1. **Conditional Statements** `if ****, else if ****, else`
2. **Loops** **Repetition of a task**

These concepts are extremely important because they develop your:

- Logical thinking
- Problem-solving ability
- Programming intuition
- Understanding of algorithms and patterns

If your fundamentals of conditions and loops are weak, advanced DSA can become difficult later.

PART A — IF / ELSE

2. What is a Conditional Statement?

Normally, a program executes statements **sequentially**:

Line 1

Line 2

Line 3

Line 4

But sometimes we don't want every line to execute.

We may want:

"Execute this code only if a particular condition is true."

That's where **conditional statements** come in.

Definition

A conditional statement decides **whether a particular piece of code should execute based on a condition**.

3. Basic if-else Logic

The basic idea is:

IF condition is TRUE

Execute IF block

IF condition is FALSE

Execute ELSE block

Example: Even / Odd

A number is even when:

```
number % 2 == 0
```

So:

```
IF n % 2 == 0
```

```
    EVEN
```

```
ELSE
```

```
    ODD
```

4. C++ / Java — if-else Syntax

C++ and Java have almost identical syntax for these concepts.

Basic Syntax

```
if (condition)
{
    // code
}
else
{
    // code
}
```

Example

```
if (n % 2 == 0)
{
    cout << "Even";
}
else
{
    cout << "Odd";
}
```

5. Curly Braces { }

Curly braces define a **block of code**.

One statement

For one statement, curly braces can technically be omitted:

```
if (condition)
    cout << "Hello";
```

Multiple statements

If you have multiple statements:

```
if (condition)
{
    cout << "Hello";
    cout << "World";
}
```

Best Practice

Even when braces aren't technically required, the lecture recommends developing the habit of **always using curly braces**.

Why?

- Better readability
 - Easier maintenance
 - Fewer mistakes
-

6. How **if-else** Executes

Consider:

```
if (condition)
{
    // IF block
}
else
{
    // ELSE block
}
```

Case 1 — Condition is TRUE

```
IF   executes
ELSE doesn't execute
```

Case 2 — Condition is FALSE

```
IF   doesn't execute
ELSE executes
```

Important

Both blocks don't execute together.

7. 0 and Non-Zero Values

In C++ and Java, the lecture explains the idea of:

```
0      FALSE
Non-zero  TRUE
```

Therefore:

```
if (1)
{
    cout << "Hello";
}
```

will execute.

Similarly:

```
if (10)
{
    cout << "Hello";
}
```

will also execute.

But:

```
if (0)
{
    cout << "Hello";
}
```

will not execute.

Easy Memory Trick

0 OFF / FALSE
Non-zero ON / TRUE

8. = VS ==

This is one of the **most important concepts**.

= Assignment Operator

Used to **assign a value**.

```
a = 10;
```

Meaning:

| Put 10 inside a .

== Comparison Operator

Used to **check equality**.

```
a == 10
```

Meaning:

| Is a equal to 10 ?

Comparison

Operator	Meaning	Example
=	Assignment	a = 10
==	Comparison	a == 10

Common Mistake

Don't confuse:

```
if (a = 10)
```

with:

```
if (a == 10)
```

The first performs an assignment, while the second checks equality.

9. else if

Sometimes there are **more than two possibilities**.

For example:

```
IF condition A
```

```
ELSE IF condition B
```

```
ELSE
```

C++ / Java

```
if (a > 10)
{
```

```
// Case 1
}  
else if (a > 5)  
{  
    // Case 2  
}  
else  
{  
    // Case 3  
}
```

10. Flow of **if** **else if** **else**

Conditions are checked **from top to bottom**.

```
Condition 1?  
  
TRUE  Execute block 1  STOP  
FALSE  
Condition 2?  
  
TRUE  Execute block 2  STOP  
FALSE  
Condition 3?  
  
TRUE  Execute block 3  STOP  
FALSE  
ELSE  Execute default block
```

Key Rule

Once one condition becomes **TRUE**, the remaining **else if** / **else** blocks are skipped.

11. Logical Operators

Sometimes one condition isn't enough.

Example:

| *A number should be **even AND divisible by 3**.*

We need to check two conditions:

```
n % 2 == 0
```

```
n % 3 == 0
```

To combine conditions, we use **logical operators**.

12. AND Operator

Meaning

ALL conditions must be TRUE.

Symbol:

```
&&
```

Example:

```
if (n % 2 == 0 && n % 3 == 0)
{
    cout << "Condition satisfied";
}
```

For the condition to be true:

```
Condition A   TRUE
AND
```

Condition B TRUE

Truth Table

ABA AND B		
T	T	T
T	F	F
F	T	F
F	F	F

Memory Trick

AND = Everything must be true.

13. OR Operator

Meaning

At least one condition must be TRUE.

Symbol:

||

Example:

```
if (n % 2 == 0 || n % 3 == 0)
{
    cout << "Condition satisfied";
}
```

Truth Table

ABA OR B		
T	T	T
T	F	T
F	T	T
F	F	F

Memory Trick

OR = At least one must be true.

14. C++ vs Java — Logical Operators

The lecture highlights an important syntax difference.

C++

C++ can use:

```
&&  
||
```

and also the English-style logical operators mentioned in the lecture.

Java

Java uses:

```
&& // AND  
|| // OR
```

The lecture specifically emphasizes remembering the **symbol form in Java**.

PART B — LOOPS

15. What is a Loop?

A **loop** is used when we want to perform the same task repeatedly.

Example

Suppose you want:

```
Hello World  
Hello World  
Hello World  
...
```

10 times.

Writing the same statement 10 times is inefficient.

Instead, use a loop.

Definition

Loop = Repetition of a block of code.

16. Why Do We Need Loops?

Without loops:

```
Print  
Print  
Print  
Print  
Print
```

...

With a loop:

Repeat this instruction 10 times

Benefits

- Saves time
 - Reduces code
 - Handles repetition
 - Makes programs more logical and scalable
-

17. Types of Loops

The lecture mentions three common loop types:

1. `for`
2. `while`
3. `do-while`

For now, the main focus is:

| `for` loop

PART C — FOR LOOP IN C++ / JAVA

18. Basic Structure

A C++ / Java `for` loop contains **three parts**:

```
for (initialization; condition; update)
{
    // code
}
```

```
}
```

Example:

```
for (int i = 0; i < 5; i++)  
{  
    cout << "Hello";  
}
```

19. The Three Parts of a **for** Loop

Part 1 — Initialization

Example:

```
int i = 0
```

This executes **once**, when the loop starts.

Purpose:

Set the initial value of the loop variable.

Part 2 — Condition / Comparator

Example:

```
i < 5
```

This determines whether the loop should execute.

Every iteration:

Condition TRUE enter loop
Condition FALSE exit loop

Part 3 — Update

Example:

```
i = i + 1
```

This executes **after each loop iteration**.

It can:

- Increase `i`
 - Decrease `i`
 - Change `i` in some other way
-

20. for Loop Execution Flow

Example:

```
for (int i = 0; i < 5; i++)  
{  
    cout << "Hello";  
}
```

Execution

```
i = 0
```

```
0 < 5   TRUE   Print
```

```
i = 1
```

```
1 < 5  TRUE  Print
```

```
i = 2
```

```
2 < 5  TRUE  Print
```

```
i = 3
```

```
3 < 5  TRUE  Print
```

```
i = 4
```

```
4 < 5  TRUE  Print
```

```
i = 5
```

```
5 < 5  FALSE
```

```
EXIT
```

Result

Hello is printed:

5 times

21. The Golden Formula for for

Remember:

INITIALIZATION

CONDITION

LOOP BODY

UPDATE

CONDITION

LOOP BODY

UPDATE

...

FALSE EXIT

PART D — PYTHON IF-ELSE

22. Python **if-else** Syntax

Python syntax is different from C++ / Java.

Python

```
if condition:  
    # code  
else:  
    # code
```

Important

Python uses:

Colon `:` + *indentation*

instead of curly braces.

23. Python Indentation

Example:

```
if n % 2 == 0:
    print("Even")
else:
    print("Odd")
```

The indented code belongs to the corresponding block.

Indentation

The lecture describes the common indentation as:

```
4 spaces
```

or one tab.

Important

Incorrect indentation can cause a syntax/structure problem.

24. Python if-elif-else

Python uses:

```
if condition:
    # block 1
elif condition:
    # block 2
else:
    # block 3
```

elif

elif means:

else if

Example:

```
if n > 10:
    print("Case 1")
elif n > 5:
    print("Case 2")
else:
    print("Case 3")
```

Only the first satisfied branch executes.

25. Python 0 and Non-Zero

The lecture explains the same general idea:

```
0      FALSE
Non-zero  TRUE
```

Example:

```
if 1:
    print("Hello")
```

runs.

But:

```
if 0:
    print("Hello")
```

doesn't enter the `if` block.

PART E — PYTHON FOR LOOP

26. Python `for` Loop

Python's `for` loop syntax differs significantly from C++ / Java.

Basic form:

```
for i in range(start, stop, jump):  
    # code
```

The three concepts are:

```
START  
STOP  
JUMP
```

27. `range()` — Start, Stop, Jump

Example:

```
for i in range(1, 5, 1):  
    print(i)
```

Values of `i` :

```
1  
2  
3  
4
```

Critical Rule

The `stop` value is **not included**.

Therefore:

```
range(1, 5)
```

means:

```
1, 2, 3, 4
```

NOT:

```
1, 2, 3, 4, 5
```

Golden Rule

Start is included; Stop is excluded.

28. Default Jump

The `jump` value is optional.

```
range(1, 5)
```

is equivalent to:

```
range(1, 5, 1)
```

So the default jump is:

1

29. Changing the Jump

Example:

```
for i in range(1, 5, 2):  
    print(i)
```

Values:

```
1  
3
```

Because the loop jumps by 2.

30. Printing Numbers 1–10

To print numbers from 1 through 10:

```
for i in range(1, 11):  
    print(i)
```

Why 11 ?

Because the stop value is excluded.

```
range(1, 11)
```

```
1 2 3 4 5 6 7 8 9 10
```

31. Printing Odd Numbers

Start at 1 and jump by 2 :

```
for i in range(1, 11, 2):  
    print(i)
```

Output:

```
1  
3  
5  
7  
9
```

32. Printing Even Numbers

Start at 2 and jump by 2 :

```
for i in range(2, 11, 2):  
    print(i)
```

Output:

```
2  
4
```

```
6
8
10
```

Pattern

```
Odd   start at 1, jump by 2
Even  start at 2, jump by 2
```

33. Python for Loop With a List

A Python `for` loop can also iterate directly through a list.

Suppose:

```
names = ["Pratyush", "Raj", "Amit"]
```

You can write:

```
for name in names:
    print(name)
```

The loop processes each element:

```
Pratyush
Raj
Amit
```

General Pattern

```
for variable in collection:  
    # use variable
```

34. Sum of Numbers Using a Loop

Suppose we want:

```
1 + 2 + 3 + 4 + 5
```

We can maintain a `sum` variable.

Logic

```
sum = 0  
  
i = 1    sum = sum + 1  
i = 2    sum = sum + 2  
i = 3    sum = sum + 3  
...
```

Python

```
sum = 0  
  
for i in range(1, 6):  
    sum = sum + i  
  
print(sum)
```

Result:

35. C++ / Java vs Python

Concept	C++ / Java	Python
if	if (condition)	if condition:
Block	{ }	Indentation
else	else	else:
Else-if	else if	elif
for	for(init; condition; update)	for i in range(...)
Loop block	{ }	Indentation
Common print	cout / System.out.println	print()

36. High-Yield Concepts

Must Remember

IF-ELSE

```
if condition is TRUE
else condition is FALSE
```

ELSE-IF

```
if
```

else if

else

Only the first satisfied branch executes.

Logical Operators

AND && all conditions must be true

OR || at least one condition must be true

Comparison

= assignment

== equality comparison

C++ / Java Loop

for (initialization; condition; update)

Remember:

Initialization once

Condition before each iteration

Update after each iteration

Python Loop

```
for i in range(start, stop, jump):
```

Remember:

```
START    included  
STOP     excluded  
JUMP     default = 1
```

37. Common Mistakes

Mistake 1 — `=` instead of `==`

```
if (a = 10)
```

For equality checking, use:

```
if (a == 10)
```

Mistake 2 — Forgetting Python `:`

```
if n > 10
```

```
if n > 10:
```

Mistake 3 — Wrong Python indentation

Keep statements belonging to a block consistently indented.

```
if condition:  
    print("Inside")
```

Mistake 4 — Python `range()` stop confusion

```
range(1, 10)
```

does **not** include `10` .

It gives:

```
1  9
```

Mistake 5 — Forgetting C++ semicolon

```
cout << "Hello";
```

Remember the `;` .

38. Practice Problems / Homework

The lecture gives **three important practice programs**.

Problem 1 — Sum of First 10 Numbers

Find:

$1 + 2 + 3 + \dots + 10$

Use a loop.

Problem 2 — Sum of First 10 Even Numbers

Find the sum of:

$2 + 4 + 6 + \dots + 20$

Use a loop.

Problem 3 — Sum of Even Numbers Between 20 and 40

Both endpoints are included.

Numbers:

20, 22, 24, 26, 28, 30,
32, 34, 36, 38, 40

Find their sum using:

- A loop
- Appropriate conditions/range

Purpose

These problems combine:

Loops + Conditions + Logical Thinking

Practicing these should strengthen your basic understanding of `if-else` and `for` loops.

39. FINAL REVISION MAP

DSA DAY 2

IF-ELSE

LOOPS

`if` `elif` `else` `for` `while` `do-while`

Conditions

Comparison

AND

OR

C++ / Java:

`if (condition) { }`

Python:

`if condition:`

 indentation

THE 10-SECOND CHEAT SHEET

IF:

 TRUE IF block

 FALSE ELSE block

`=` assign

`==` compare

AND && ALL true

OR || ANY true

C++/Java FOR:

for (init; condition; update)

Python FOR:

for i in range(start, stop, jump)

Python:

STOP is excluded

JUMP defaults to 1

Loop:

REPEAT a task efficiently

Core Takeaway

Conditions control WHAT code executes.

Loops control HOW MANY TIMES code executes.

Together, they form one of the most important foundations for **DSA problem-solving and algorithmic thinking**. DSA BASICS — DAY 3

FUNCTIONS — C++ • Java • Python

*Core Idea: A function is a reusable block of code that takes input processes it
returns output.*

1. Why Are Functions Important?

Functions are extremely important in DSA because platforms such as **GeeksforGeeks and LeetCode** generally don't ask you to write the entire program.

Instead, they provide the function structure and ask you to:

Implement the logic inside the function.

So, understanding functions is essential before moving deeper into DSA patterns.

Typical DSA Problem

You may be given something like:

```
int findSum(int n)
{
    // Your logic
}
```

Your job is primarily to write the **logic** that belongs inside the function.

2. What is a Function?

Think of a function as a **black box** .

INPUT

FUNCTION

Processing
/ Logic

OUTPUT

Example

Suppose a function doubles a number:

Input: 10

Function

Output: 20

Another example:

Input: 5

double()

Output: 10

The function hides the internal implementation from the person using it.

3. Function = Input Processing Output

A simple mental model:

FUNCTION

Input

Processing / Logic

Output / Return

For example:

Input: n

Logic: $n \times 2$

Output: $2n$

This model will remain useful throughout DSA.

4. Three Main Parts of a Function

Especially in **C++ and Java**, a function can be understood through three major components:

Return Type

What type of value will the function return?

Examples:

```
int  
string  
char  
double  
...
```

Function Name

What is the function called?

Example:

```
doubleNumber
```

Input / Parameters

What information does the function need?

Example:

```
int n
```

Function Structure

Return Type

Function Name

Parameters

Body

Return

5. C++ Function Syntax

A basic C++ function:

```
int doubleNumber(int n)
{
    int result = 2 * n;
    return result;
}
```

Let's break it down:

int

Return type

doubleNumber

Function name

```
(int n)
```

Input / parameter

```
{  
    ...  
}
```

Function body

```
return result;
```

Output

6. Return Type

The **return type** tells the compiler what kind of value the function will return.

Integer

```
int doubleNumber(int n)
```

The function returns an integer.

String

Conceptually:

```
string getName(...)
```

The function returns a string.

Other Types

A function can return different kinds of data depending on the language and implementation.

The key idea is:

Return type = Type of output produced by the function.

7. Function Name

Every function needs a name so that you can **call** it later.

Example:

```
int doubleNumber(int n)
```

Here:

```
doubleNumber
```

is the function name.

When you want to use the function, you refer to this name.

Important

The function name must be used correctly.

For example, if the function is:

```
doubleNumber()
```

calling:

```
doubleNum()
```

would refer to a different name.

8. Parameters / Arguments

These two terms are closely related.

Parameter

The variable defined in the function declaration:

```
int doubleNumber(int n)
```

Here:

```
n    parameter
```

Argument

The actual value supplied when calling the function:

```
doubleNumber(10)
```

Here:

```
10   argument
```

Easy Memory Trick

Function definition:

```
int n
```

Parameter

Function call:

```
doubleNumber(10)
```

Argument

9. Number of Inputs

A function isn't restricted to one input.

It can take:

0 inputs

1 input

2 inputs

3 inputs

...

many inputs

Example:

```
int sum(int a, int b)
{
    return a + b;
}
```

Here the function takes:

2 inputs

10. Input Types Matter

In languages such as C++ and Java, the type of every parameter matters.

For example:

```
int function(int a, string b)
```

means:

```
1st input  integer  
2nd input  string
```

The expected input structure must be respected when calling the function.

Order Matters

If a function expects:

```
int  string
```

you shouldn't treat it as:

```
string  int
```

unless the function has been designed to accept that arrangement.

11. Function Body

The function body contains the **actual implementation / logic**.

Example:

```
int doubleNumber(int n)
{
    int result = 2 * n;

    return result;
}
```

The body is:

```
{
    int result = 2 * n;
    return result;
}
```

This is where the function performs its work.

12. return Statement

The `return` statement sends the result back to the caller.

Example:

```
return result;
```

Suppose:

```
n = 10
```

Then:

```
result = 2 × 10  
      = 20
```

The function returns:

```
20
```

13. How to Call a Function

Creating a function isn't enough.

You also need to **call/use** it.

Suppose:

```
int doubleNumber(int n)  
{  
    return 2 * n;  
}
```

You can call it like:

```
int m = 10;  
  
int result = doubleNumber(m);
```

Flow

```
m = 10  
  
doubleNumber(m)
```

n receives the value 10

$2 \times n$

20

result = 20

14. Function Reusability

This is one of the **main reasons functions exist**.

Imagine you need the same calculation **100,000 times**.

Without a function:

Write the same logic

again

again

again

100,000 times

That's terrible for:

- Code size
- Development time
- Maintenance
- Readability
- Debugging

With a function:

Write logic once

Call function whenever required

Reuse the logic

Core Principle

Functions provide code reusability.

15. Real-World Example — Google Maps

The lecture uses the idea of a graph algorithm such as **Dijkstra's algorithm**.

Suppose a system repeatedly needs to calculate distances between points.

Instead of rewriting the entire algorithm every time:

100+ lines of algorithm

rewrite again

rewrite again

...

developers can place the logic inside a function.

Then they can simply call that function whenever required.

```
distanceFunction(...)
```

Result

A complex implementation becomes reusable through a simple function call.

16. Function as an Abstraction

The person calling the function doesn't necessarily need to know how the internal logic works.

For example:

```
doubleNumber(10);
```

You don't need to rewrite:

```
multiply by 2  
calculate result  
return result
```

every time.

The function handles it.

This gives:

Abstraction + Reusability

17. Important: Parameter and Local Variable Are Separate

Consider:

```
int doubleNumber(int n)  
{  
    return 2 * n;  
}
```

```
}
```

Then:

```
int n = 10;  
doubleNumber(n);
```

The `n` outside and the `n` inside the function should be understood as **separate variables in their respective scopes**.

The lecture notes that more detailed concepts such as:

- Pass by value
- Pass by reference
- References

will be covered later.

For now:

Don't assume two variables are the same merely because they have the same name.

18. You Don't Always Need to Store the Return Value

Suppose:

```
int doubleNumber(int n)  
{  
    return 2 * n;  
}
```

You can store the answer:

```
int result = doubleNumber(10);
```

Or use the returned value directly:

```
cout << doubleNumber(10);
```

Why does this work?

Because the function returns a value.

That value can then be:

- Stored in a variable
 - Printed
 - Used in another calculation
 - Passed somewhere else
-

19. Function Calling Another Function

Functions can call other functions.

Conceptually:

```
main()
```

```
function A()
```

```
function B()
```

```
function C()
```

This allows large programs to be broken into smaller reusable components.

The lecture also mentions that a function can potentially call itself—this idea becomes important later when studying **recursion**.

20. `main()` Is Also a Function

Look at:

```
int main()
{
    ...
}
```

This has the same general structure:

```
return type  int
function name main
parameters  ()
body  {}
```

Therefore:

`main()` is a function.

C++ / Java

Program execution begins from `main()` .

Python

The execution model is different; Python doesn't require a `main()` function in the same way.

PART B — JAVA

21. Java Function Syntax

Java functions are similar conceptually to C++ functions.

A basic structure is:

```
public static int doubleNumber(int n)
{
    int result = 2 * n;
    return result;
}
```

The important parts are:

```
public static
int
doubleNumber
(int n)
{ body }
```

22. **public static** in Java

For the current level of the course, the lecture recommends remembering:

```
public static
```

before the function.

A deeper explanation of:

- Classes
- Objects
- **static**
- Object-oriented programming

will come later.

For now:

*Focus on the **function structure and logic**, not OOP details.*

23. Java `main()` Function

Java also has a `main()` function:

```
public static void main(String[] args)
{
    ...
}
```

This is the starting point of a Java program.

Notice the similarity:

```
public static int doubleNumber(...)
```

and:

```
public static void main(...)
```

Both are functions/methods with a return type, name, parameters, and body.

PART C — DSA FUNCTIONS

24. Functions on GFG / LeetCode

This is **VERY IMPORTANT for DSA**.

On coding platforms, you are often given a function such as:

```
int findSum(int n)
{
    // Implement logic
}
```

You don't need to write:

- Header files
- `main()`
- Input handling
- Output boilerplate

The platform handles much of that.

Your responsibility:

Understand the function signature and implement its logic correctly.

25. How to Read a DSA Function

Suppose you're given:

```
int findSum(int n)
```

Read it from left to right:

`int`

Return type = integer

`findSum`

Function name

```
int n
```

One integer input

Therefore:

The function accepts an integer and must return an integer.

26. Example DSA Problem — Sum of Natural Numbers

Problem

Given an integer `n`, calculate:

$$1 + 2 + 3 + \dots + n$$

If:

$$n = 5$$

then:

$$1 + 2 + 3 + 4 + 5 = 15$$

Special case:

$$n = 0$$

Then:

```
sum = 0
```

27. Step 1 — Understand the Function Signature

Suppose GFG gives:

```
int findSum(int n)
```

Identify:

Part	Meaning
int	Return type
findSum	Function name
int n	Input parameter

So we know:

```
Input   integer n
Output  integer sum
```

28. Step 2 — Handle the Special Case

The problem says:

| If `n = 0`, return `0`.

So:

```
if (n == 0)
{
    return 0;
}
```

Remember

Use:

```
==
```

for comparison.

Not:

```
=
```

29. Step 3 — Find the Mathematical Formula

The sum of natural numbers from 1 to n is:

$$\text{Sum} = \frac{n(n+1)}{2}$$

So:

```
int result = n * (n + 1) / 2;
```

Then:

```
return result;
```

Complete Logic

```
int findSum(int n)
{
    if (n == 0)
    {
        return 0;
    }

    int result = n * (n + 1) / 2;

    return result;
}
```

30. DSA Problem-Solving Method

When given a function-based DSA question:

Step 1

Read the problem carefully.

Step 2

Understand the function signature.

Step 3

Identify:

```
Input?
Output?
```

Return type?

Step 4

Find the required logic/algorithm.

Step 5

Handle special cases.

Step 6

Implement the logic inside the function.

Step 7

Return the answer.

31. Function Signature — Quick Decoder

Given:

```
int findSum(int n)
```

Think:

int

What comes out?

findSum

What is it called?

int n

What goes in?

Result

INPUT int

OUTPUT int

NAME findSum

32. C++ vs Java — Functions

Feature	C++	Java
Return type	int	int
Function name	doubleNumber	doubleNumber
Parameters	(int n)	(int n)
Body	{ }	{ }
Return	return result;	return result;
Common declaration	int doubleNumber(...)	public static int doubleNumber(...)
Starting point	main()	main()

33. C++ / Java Function Flow

CALL FUNCTION

PASS ARGUMENT

PARAMETER RECEIVES VALUE

FUNCTION BODY EXECUTES

LOGIC PROCESSES INPUT

RETURN RESULT

CALLER RECEIVES RESULT

Example

```
doubleNumber(10)
```

```
n = 10
```

```
2 × n
```

```
20
```

```
return 20
```

34. High-Yield Vocabulary

Term	Meaning
Function	Reusable block of code
Return type	Type of value returned
Function name	Identifier used to call function
Parameter	Variable defined in function input
Argument	Actual value passed to function

Function body	Code containing the implementation
<code>return</code>	Sends value back to caller
Function call	Using/executing a function
Reusability	Ability to use same logic multiple times

35. Common Mistakes

Mistake 1 — Forgetting the return

If the function is:

```
int doubleNumber(int n)
```

it is expected to return an integer.

Don't forget:

```
return result;
```

Mistake 2 — Wrong parameter type

If:

```
int doubleNumber(int n)
```

expects an integer, understand what type of argument should be passed.

Mistake 3 — Confusing parameter and argument

```
int doubleNumber(int n)
```

`n` parameter

```
doubleNumber(10)
```

`10` argument

Mistake 4 — Writing the entire program on DSA platforms

If the platform already gives:

```
int findSum(int n)
{
    // ...
}
```

you generally need to implement the requested logic rather than rebuilding the entire program.

Mistake 5 — Confusing `=` and `==`

```
n = 0
```

assignment

```
n == 0
```

comparison

36. Ultimate Mental Model

Whenever you see a function, ask **4 questions**:

What goes IN?

Parameters / inputs

What happens INSIDE?

Logic / processing

What comes OUT?

Return value

Why is it useful?

Reusability + abstraction

37. One-Page Revision Sheet

FUNCTION

INPUT

FUNCTION

PROCESSING

RETURN

C++

```
int functionName(int n)
{
    // logic

    return result;
}
```

Java

```
public static int functionName(int n)
{
    // logic

    return result;
}
```

Key Parts

```
Return Type
+
Function Name
+
Parameters
+
Function Body
+
Return
```

Calling

```
int result = functionName(value);
```

or:

```
cout << functionName(value);
```

DSA Platform

Given Function

Read Signature

Understand Input

Understand Output

Find Logic

Handle Edge Cases

Return Answer

FINAL TAKEAWAY

Remember this single sentence:

A function is a reusable black box that accepts inputs, performs some logic, and returns an output.

And for DSA:

Don't rush to code. First decode the function signature: WHAT goes in, WHAT must come out, and WHAT logic connects them.

Once you become comfortable with this, **GFG/LeetCode-style problems become much easier to approach**, because the platform has already provided much of the program—you mainly need to solve the logic inside the function. DSA BASICS — DAY 4

ARRAYS & LINKED LISTS

The Final Day of DSA Basics

Day 4 Basics — Patterns + Algorithms

1. Goal

DSA Basics

:

- Variables
- Input / Output
- Printing
- Functions
- If-Else
- Loops

topics:

Arrays

Linked Lists

fundamental data structures

complex data structures foundation

2. Data Structure

?

Simple Definition

Data Structure = structure data store data

data organize :

- Store
- Access
- Modify
- Process

Data Structure

3. Variable = Simplest Data Structure

variables

Variable **box** imagine

10

box value

:

10
20
"Ram"

variable value

box :

10

:

20

, value replace

:

Variable

Box

Value

simple data structure

4. Array & Linked List

?

imagine :

Box

Box

Box

Box

Box

...

boxes

boxes organized ,
useful data structures

:

5. Data Structures Foundation

Lecture 5, covering the foundation of complex data structures. This lecture covers the foundation of data structures, starting with Array and Linked List.

Arrays :

- Stack
- Queue
- Deque

Linked Lists :

- HashMap
- Sets
- Trees
- Graphs

Main Idea

Array / Linked List

Fundamental Data Structures

More Complex Data Structures

Arrays Linked Lists

PART A — ARRAYS

6. Array ?

Variable :

| *box*

Array :

| *boxes* , *line* *organize*

Conceptually:

10 20 30 40 50 60

boxes continuous arrangement

Definition

| *Array = Similar type elements continuous allocation/arrangement.*

7. Array Index

Array element **position**

position :

| *Index*

Important Rule

| *Array indexing 0 , 1*

Example:

Index: 0 1 2 3 4

Value: 10 20 30 40 50

:

Index 0 10

Index 1 20

Index 2 30

Index 3 40

Index 4 50

8. Index = Position

Index :

| *Element position/*

Example

:

| *Array first element ?*

:

First element index 0

:

| *Array fourth element ?*

:

Fourth element index 3

Formula

1st element index 0

2nd element index 1

3rd element index 2

4th element index 3

:

Nth element index N - 1

9. Array Size

Example:

Index:

0 1 2 3 4 5 6 7

:

8 boxes

:

Array size = 8

Last index:

7

size:

8

indexing 0

10. Same Type Elements

Array generally **similar type** **elements**

integer array :

10 20 30 40 50

elements integers

:

10

20

30

"Ram"

50

Audio File

Video

conceptually:

Integer Array

10 20 30 40 50

Key Point

Array *data* *organized*

Array *data* *organized*

11. Array Best Intuition — Classroom Example

Imagine school

Class 9 students classroom :

CLASS 9

Student Student ...

Student Student ...

Principal Class 9 students

student :

"Class 9?"

room Class 9 students

array basic intuition

Array:

organized/continuous structure

Similar elements

Direct access

12. Array Strength — Random Access

Array strength:

Random Access

array particular element , index

Example:

Array:

Index 0 1 2 3 4 5

Value 10 20 30 40 50 60

:

Fourth element?

:

index = 3

:

0 1 2 3

element practically search

13. Roll Number Example

Class students roll numbers :

```
1 Student A
2 Student B
3 Student C
...
40 Student X
```

:

"Roll number 40 student "

student 40 identify

intuition array **random access**

14. Array Limitation — Middle Insertion

Array **end** **insertion** relatively

Example:

```
10 20 30 40 _ _ _ _
```

50 add :

```
10 20 30 40 50 _ _ _
```

:

60
70

15. Middle Insertion Costly ?

:

10 20 30 40 50 60 70 _

80

40

insert

Array continuous

existing elements shift :

10 20 30 40 50 60 70

Shift elements

Conceptually:

70
60
50
40 ...

80 space

Final arrangement:

10 20 30 40 80 50 60 70

Why costly?

elements :

position *position* *shift*

16. Array Middle Deletion

Same problem deletion

Suppose:

10 20 30 40 50 60 70

40 delete

gap

elements shift :

10 20 30 50 60 70

:

Middle insertion/deletion *shifting*

shifting operation costly

17. Array Strengths vs Weaknesses

OperationArray	
----------------	--

Random Access	Excellent
End Insertion	
End Deletion	
Middle Insertion	Costly
Middle Deletion	Costly

Array Core Nature

Fast Access + Costly Middle Modification

PART B — LINKED LIST

18. Linked List ?

Linked List boxes example

Computer memory locations

spaces continuous :

Position:
1 20 30 50

boxes line adjacent

Problem

Array continuous arrangement

spaces scattered ?

Linked List useful

19. Linked List Basic Idea

Linked List elements memory continuous positions

:

element

element

location

information

Conceptually:

Node 1

Node 2

Node 3

Node 4

NULL

Example:

[10 | next] [20 | next] [30 | next] [50 | NULL]

20. Classroom / Chair Example

chairs available :

Chair 1

Chair 20

Chair 30

Chair 50

continuous

students

chairs

Linked List :

Student 1

Chair 1

:

| "Student 2 Chair 20"

Student 2

Chair 20

:

| "Student 3 Chair 30"

Student 3

Chair 30

:

| "Student 4 Chair 50"

Student 4

Chair 50

student :

NULL

21. Linked List Structure

HEAD

[10 | next] [20 | next] [30 | next] [50 | NULL]

Important Terms

Head

node

Next

node location/reference information

NULL

node

Lecture

head, pointers

detailed implementation

detail

22. Linked List Traversal

Suppose:

HEAD

10 20 30 50 NULL

30

30 jump

?

:

10 20
20 30

30

20

:

HEAD

10

20

30

-

23. Linked List Weakness — Random Access

Linked List

limitation

Array :

arr[index]

direct access conceptually possible

Linked List :

initially *Head*

:

Head

Next

Next

Next

Example

100 nodes 50th node :

1 2 3 ... 50

sequence

Conclusion

Linked List random access costly

24. Linked List Strength — Middle Insertion

Linked List

Suppose:

10 20 30

20

30

element 40 insert

node :

40

links adjust :

10 20 40 30

:

1. Previous node next
2. New node next set

Key Idea

Existing elements shift

25. Linked List Insertion Intuition

:

20 30

node:

40

:

20 40 30

links

array :

```
30  shift
40  shift
50  shift
...
```

26. Linked List Deletion

Suppose:

```
10  20  30  40
```

```
30
```

```
20      :
```

```
"      next 30      , 40      "
```

```
:
```

```
10  20  40
```

```
30 chain
```

Core Idea

```
Deletion      links      , elements      shift
```

27. Array vs Linked List

- compare

Feature	Array	Linked List
Storage arrangement	Continuous	Non-continuous
Random Access		Costly
Middle Insertion	Costly	
Middle Deletion	Costly	
Elements Shift	required	Generally
Starting information	Index-based access	Head
Main Strength	Fast access	Easy insertion/deletion

28. The Core Trade-Off

topic simple trade-off :

ARRAY

Fast Random Access

Middle Modification
Costly

LINKED LIST

Random Access Costly

Middle Modification
Easy

Line

Array

Access

, Linked List

Modification

.

29. Opposite Strengths

Array :

Random Access EASY

Middle Insert/Delete HARD

Linked List :

Random Access HARD

Middle Insert/Delete EASY

structures

important conceptual difference

30. Real-World Analogy

Array = Classroom

Students organized classroom

Student 1

Student 2

Student 3

Student 4

Student 5

particular position

student insert/remove arrangement maintain
shifting/rearrangement

Linked List = Connected Students

student :

“ ?”

Student 1

Student 2

Student 3

Student 4

particular student

chain follow

student links

31. Why Both Are Fundamental?

Array Linked List - problems efficiently handle

Array

Useful when:

Frequent/random access

Linked List

Useful when:

Frequent insertion/deletion ,

universally “best”

Data Structure Choice Depends On:

problem

operation

32. Future DSA Connection

Lecture , :

- Trees
- Graphs
- HashMap
- Sets
- Stack
- Queue
- Deque

, Arrays Linked Lists understanding -

conceptual foundation

important

33. 1D Array

array :

10 20 30 40 50

$$\vdots$$

1D Array

```
line      imagine      :
```

[10][20][30][40][50]

34. 2D Array

Array dimension

2D Array

matrix :

```
[ 0 1 2 3 ]  
[ 4 5 6 7 ]  
[ 8 9 10 11 ]
```

conceptually:

| *Arrays* *Arrays*

Structure

2D Array

Array 1

Array 2

Array 3

arrays organize , larger **2D array**

35. Syntax ?

lecture intentionally:

| *Concept-focused*

C++, Java Python Arrays / Linked Lists syntax implementations

Examples:

```
C++  arrays / vectors
Java  arrays / ArrayList
Python  lists
```

Linked Lists implementation language

	:	
<i>syntax</i>		<i>concept</i>
actual DSA questions	,	implementation syntax

36. Linked List Pointers

Linked List implementation **pointers/references**

implementation	-	languages
Lecture	:	
<i>Linked List</i>		<i>deeply implement</i>
Pointers detail	,	Linked List properly
implement meaningful		

37. Homework

specific homework question	
:	

Arrays Practice
Array syntax basic operations practice

Linked List

Lecture explicit advice:

| *Linked List* *deeply*

pointers detail

38.

?

DSA Basics complete

phase :

BASICS

PATTERNS

ALGORITHMS

DSA PROBLEM SOLVING

Focus:

- Patterns
- Algorithms
- Coding approach
- Better code writing
- Problem solving

actual DSA journey .

39. MASTER REVISION — DAY 4

DATA STRUCTURE

Data store manipulate organized structure.

VARIABLE

1 Box

1 Value

ARRAY

Multiple Boxes

Continuous Arrangement

Similar Type Data

Strength

Random Access

Weakness

Middle Insertion / Deletion

LINKED LIST

Node Node Node Node

node node information

Strength

Middle Insertion / Deletion

Weakness

Random Access

40. MOST IMPORTANT COMPARISON

ARRAY

Random Access

Insert/Delete Middle
COSTLY

LINKED LIST

Random Access
COSTLY

Insert/Delete Middle
EASY

Golden Rule

Array Access Fast

Linked List Modification Easy

41. Interview/DSA Memory Table

Concept	Array	Linked List
Basic idea	Continuous boxes	Connected nodes
Index	Yes	No direct index-based access
Random access	Fast	Costly
Middle insertion	Costly	Easy
Middle deletion	Costly	Easy
Shifting	Required in array-style middle modification	Generally not required
Starting point	Index	Head
Next information	Not needed	Important
Main advantage	Direct access	Flexible linking

42. 30-Second Revision

If you remember only **5 things**, remember these:

1 Variable

One box one value

2 Array

Many similar boxes arranged continuously

3 Array Index

Starts from 0

4 Array Strength

Random access is fast

5 Linked List Strength

Insertion/deletion in the middle can be easy because links are changed instead of shifting all elements

FINAL TAKEAWAY

The entire Day 4 can be reduced to one idea:

DATA STRUCTURES	
ARRAY	LINKED LIST
Fast Random Access	Easy Modification
Middle Insert/Delete Costly	Random Access Costly

And the bigger picture:

Arrays	Linked Lists	DSA	fundamental building blocks	.
goal			implementation	
:				

Data	organized	Access	Insert/Delete
operation		structure	
Basics	phase		Patterns Algorithms